

**LAPORAN WORKSHOP SISTEM INFORMASI
WEB SERVER
(TIF330805)**

SEMESTER III



BKPM: ACARA 10

**NAMA:
SHERYN FEBRYLIA HENDRITA
(E41250604)**

**PROGRAM STUDI DILUAR KAMPUS UTAMA (PSDKU)
TEKNIK INFORMATIKA KAB. NGANJUK
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI JEMBER
TAHUN 2026**

1. PENDAHULUAN

Praktikum Acara 10 bertujuan untuk memahami penerapan konsep **inheritance atau pewarisan** dalam pemrograman berorientasi objek pada aplikasi web berbasis MVC.

Pada praktikum ini dibuat class dasar atau **Base Class** yang dapat digunakan kembali oleh beberapa class turunan. Class BaseController digunakan sebagai class dasar untuk kebutuhan Controller, sedangkan BaseModel digunakan sebagai class dasar untuk Repository yang membutuhkan koneksi database.

Penerapan Base Class bertujuan untuk mengurangi penulisan kode yang berulang serta membuat struktur aplikasi menjadi lebih terorganisasi dan mudah dikembangkan.

2. HASIL PRAKTIKUM

1. Persiapan Project

1. Struktur Project Acara 10

Project Acara 10 memiliki struktur MVC dengan beberapa bagian utama, yaitu Core, Controllers, Models, Repositories, Views, config, public, dan routes.

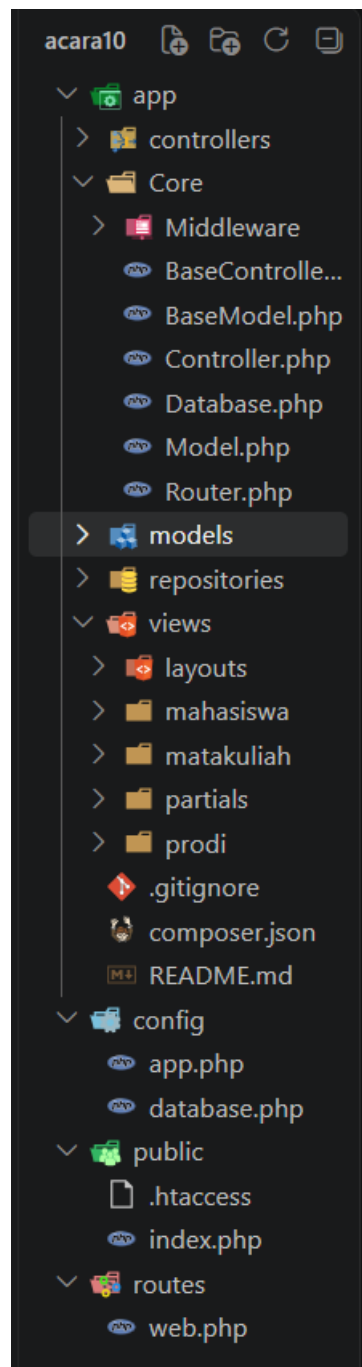
Struktur project yang digunakan adalah:

```
acara10/
├── app/
│   ├── controllers/
│   ├── Core/
│   │   ├── BaseController.php
│   │   ├── BaseModel.php
│   │   ├── Controller.php
│   │   ├── Database.php
│   │   └── Middleware/
│   ├── models/
│   └── repositories/
```

```
graph LR; Core[Core] --- views[views/]; Core --- config[config/]; Core --- public[public/]; Core --- routes[routes/];
```

Struktur tersebut menunjukkan bahwa class dasar ditempatkan pada folder Core, sedangkan Controller dan Repository menggunakan class tersebut sebagai dasar.

Gambar 1. Struktur folder project Acara 10



2. Pembuatan BaseController

BaseController merupakan class dasar yang digunakan untuk menyediakan fungsi umum bagi Controller.

Pada project Acara 10, BaseController memiliki dua method utama, yaitu view() dan redirect().

```
class BaseController
```

```

{
    protected function view(string $view, array $data = []): void
    {
        extract($data);

        require __DIR__ . '/../Views/' . $view . '.php';
    }

    protected function redirect(string $url): void
    {
        header("Location: {$url}");
        exit;
    }
}

```

Method view() digunakan untuk menampilkan file View dan mengirimkan data dari Controller ke View.

Method redirect() digunakan untuk mengarahkan pengguna ke URL tertentu setelah proses seperti tambah, edit, atau hapus data selesai.

Gambar 2. Class BaseController

```

app > Core > BaseController.php > ...
1  <?php
2
3  2 references | 4 implementations
4  class BaseController
5  {
6      9 references | 0 overrides
7      protected function view(string $view, array $data = []): void
8      {
9          extract($data);
10         require __DIR__ . '/../Views/' . $view . '.php';
11     }
12
13     9 references | 0 overrides
14     protected function redirect(string $url): void
15     {
16         header("Location: {$url}");
17         exit;
18     }
19 }

```

3. Penerapan Inheritance pada Controller

Inheritance diterapkan dengan membuat Controller mewarisi fungsi dari BaseController.

Pada project terdapat:

```
class Controller extends BaseController
{
}
```

Dengan demikian, class Controller memperoleh method yang dimiliki BaseController.

BaseController



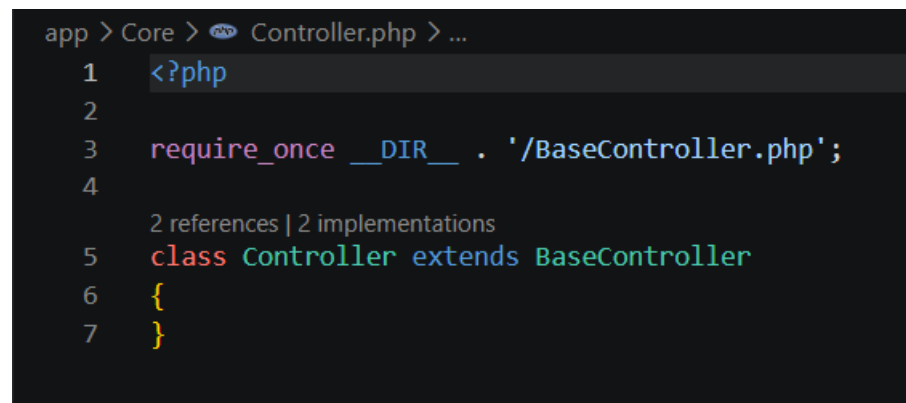
Controller



Controller lainnya

Konsep tersebut membuat fungsi umum seperti view() dan redirect() tidak perlu dibuat berulang kali pada setiap Controller.

Gambar 3. Controller mewarisi BaseController



```
app > Core > Controller.php > ...
1  <?php
2
3  require_once __DIR__ . '/BaseController.php';
4
5  2 references | 2 implementations
6  class Controller extends BaseController
7  {
8  }
```

4. MahasiswaController sebagai Class Turunan

Pada bagian mahasiswa, MahasiswaController langsung menggunakan BaseController sebagai parent class.

```
class MahasiswaController extends BaseController
{
    private MahasiswaRepository $repository;
```

Dengan inheritance tersebut, MahasiswaController dapat menggunakan method view() dan redirect() yang didefinisikan pada BaseController.

Contohnya pada method index():

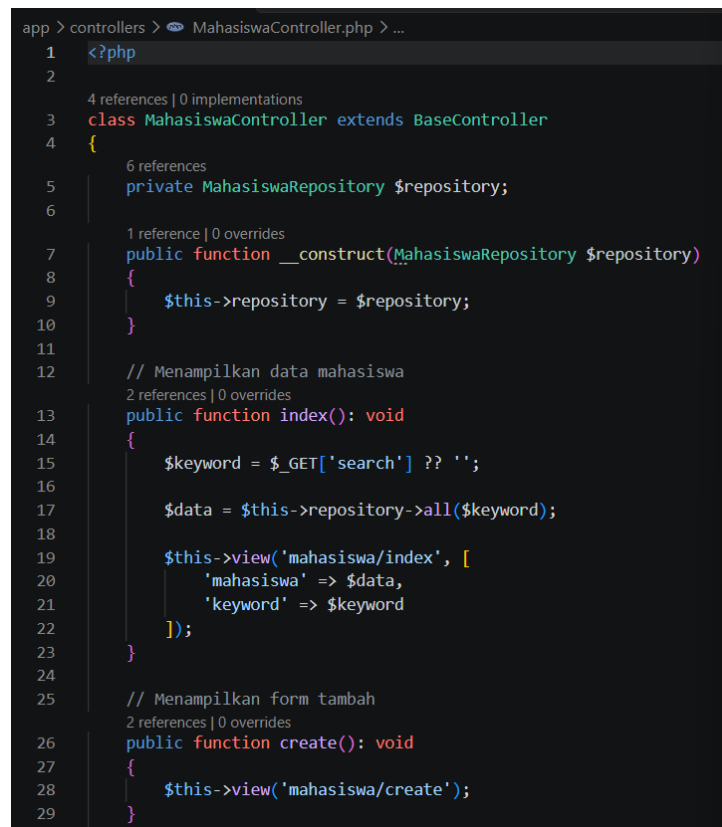
```
$this->view('mahasiswa/index', [  
    'mahasiswa' => $data,  
    'keyword' => $keyword  
]);
```

Sedangkan setelah proses penyimpanan data selesai digunakan:

```
$this->redirect('/bkpm-webserver/acara10/public/mahasiswa');
```

Method tersebut tidak perlu ditulis ulang di dalam MahasiswaController karena sudah diwariskan dari BaseController.

Gambar 4. Inheritance pada MahasiswaController



```
app > controllers > MahasiswaController.php > ...  
1 <?php  
2  
3 4 references | 0 implementations  
class MahasiswaController extends BaseController  
4 {  
5     6 references  
private MahasiswaRepository $repository;  
6  
7     1 reference | 0 overrides  
public function __construct(MahasiswaRepository $repository)  
8 {  
9     $this->repository = $repository;  
10 }  
11  
12 // Menampilkan data mahasiswa  
13 2 references | 0 overrides  
public function index(): void  
14 {  
15     $keyword = $_GET['search'] ?? '';  
16  
17     $data = $this->repository->all($keyword);  
18  
19     $this->view('mahasiswa/index', [  
20         'mahasiswa' => $data,  
21         'keyword' => $keyword  
22     ]);  
23 }  
24  
25 // Menampilkan form tambah  
26 2 references | 0 overrides  
public function create(): void  
27 {  
28     $this->view('mahasiswa/create');  
29 }  
30
```

5. Pembuatan BaseModel

Selain Base Controller, pada Acara 10 dibuat BaseModel.

BaseModel digunakan sebagai class dasar yang menyimpan koneksi PDO.

```
class BaseModel
{
    protected PDO $pdo;

    public function __construct(PDO $pdo)
    {
        $this->pdo = $pdo;
    }
}
```

Property \$pdo dibuat protected, sehingga dapat digunakan oleh class yang mewarisi BaseModel.

Constructor menerima object PDO sebagai dependency.

Gambar 5. Class BaseModel



6. MahasiswaRepository Mewarisi BaseModel

MahasiswaRepository menggunakan inheritance dengan mewarisi BaseModel.

Kode yang digunakan:

```
class MahasiswaRepository extends BaseModel
{
```

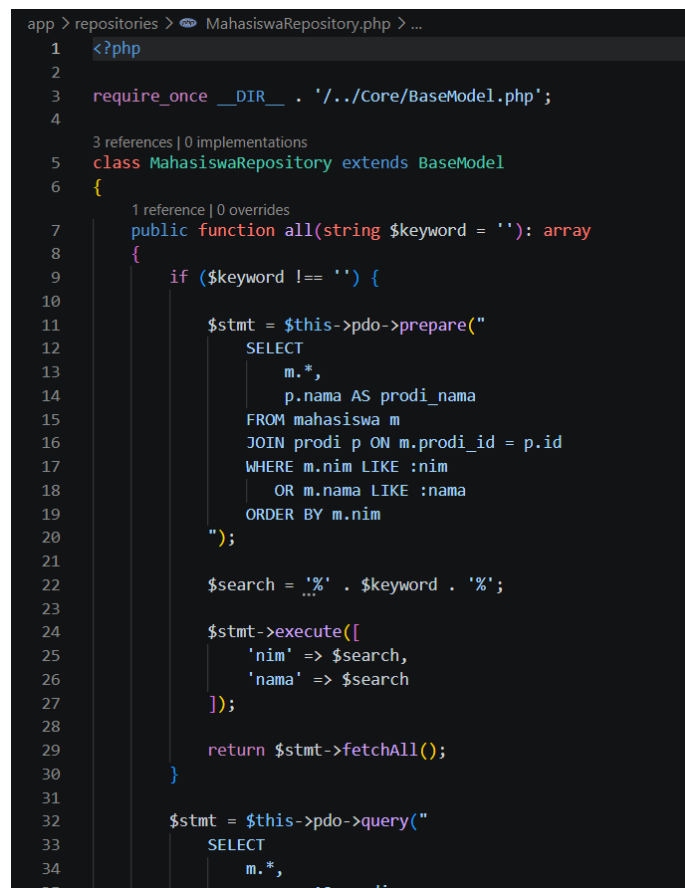
Dengan inheritance tersebut, MahasiswaRepository tidak perlu membuat property \$pdo dan constructor PDO sendiri karena sudah tersedia pada BaseModel.

Repository dapat langsung menggunakan:

`$this->pdo`

untuk menjalankan query database.

Gambar 6. MahasiswaRepository extends BaseModel



```
app > repositories > MahasiswaRepository.php > ...
1  <?php
2
3  require_once __DIR__ . '/../Core/BaseModel.php';
4
5  3 references | 0 implementations
class MahasiswaRepository extends BaseModel
6  {
7      1 reference | 0 overrides
    public function all(string $keyword = ''): array
    {
8          if ($keyword !== '') {
9              $stmt = $this->pdo->prepare("
10                 SELECT
11                     m.*,
12                     p.nama AS prodi_nama
13                 FROM mahasiswa m
14                 JOIN prodi p ON m.prodi_id = p.id
15                 WHERE m.nim LIKE :nim
16                     OR m.nama LIKE :nama
17                 ORDER BY m.nim
18             ");
19
20             $search = '%' . $keyword . '%';
21
22             $stmt->execute([
23                 'nim' => $search,
24                 'nama' => $search
25             ]);
26
27             return $stmt->fetchAll();
28         }
29
30         $stmt = $this->pdo->query("
31             SELECT
32                 m.*,
33                 p.nama AS prodi_nama
34         ");
```

7. Dependency Injection pada MahasiswaRepository

Pada public/index.php, object PDO diperoleh dari Database.

```
$database = new Database();
```

```
$pdo = $database->getConnection();
```

Kemudian PDO diberikan kepada MahasiswaRepository:

```
$mahasiswaRepository = new MahasiswaRepository($pdo);
```

Repository menerima dependency tersebut melalui constructor yang diwarisi dari BaseModel.

Alur dependency yang digunakan adalah:

Database



PDO



BaseModel



MahasiswaRepository

Dengan cara tersebut, Repository dapat menggunakan koneksi database tanpa membuat koneksi database baru sendiri.

Gambar 7. Dependency Injection PDO ke MahasiswaRepository

```
18
19  $database = new Database();
20
21  $pdo = $database->getConnection();
22
23
24  // =====
25  // MAHASISWA
26  // =====
27
28  $mahasiswaRepository = new MahasiswaRepository($pdo);
29
```

8. MahasiswaController Menerima Repository

Setelah Repository dibuat, object Repository diberikan kepada MahasiswaController.

Pada public/index.php terdapat:

```
$controller = new
```

```
MahasiswaController($mahasiswaRepository);
```

Kemudian pada MahasiswaController terdapat constructor:

```
public function __construct(MahasiswaRepository $repository)
```

```
{
    $this->repository = $repository;
}
```

Dengan demikian, Controller tidak membuat Repository secara langsung. Repository diberikan dari luar melalui constructor.

Gambar 8. Dependency Injection pada MahasiswaController

```
27
28 $mahasiswaRepository = new MahasiswaRepository($pdo);
29
30 $controller = new MahasiswaController($mahasiswaRepository);
31
32
```

```
4 references | 0 implementations
class MahasiswaController extends BaseController
{
    6 references
    private MahasiswaRepository $repository;

    1 reference | 0 overrides
    public function __construct(MahasiswaRepository $repository)
    {
        $this->repository = $repository;
    }
}
```

9. CRUD pada MahasiswaRepository

MahasiswaRepository menangani proses CRUD data mahasiswa.

Method yang tersedia adalah:

all()

find()

create()

update()

delete()

Method all() digunakan untuk mengambil seluruh data mahasiswa.

Method `find()` digunakan untuk mengambil satu data berdasarkan ID.

Method `create()` digunakan untuk menambahkan data.

Method `update()` digunakan untuk mengubah data.

Method `delete()` digunakan untuk menghapus data.

Gambar 9. Method CRUD pada MahasiswaRepository

```
app > repositories > MahasiswaRepository.php > ...
5  class MahasiswaRepository extends BaseModel
7  public function all(string $keyword = ''): array
8  {
9      if ($keyword !== '') {
10
11          $stmt = $this->pdo->prepare("
12              SELECT
13                  m.*,
14                  p.nama AS prodi_nama
15              FROM mahasiswa m
16              JOIN prodi p ON m.prodi_id = p.id
17              WHERE m.nim LIKE :nim
18                  OR m.nama LIKE :nama
19              ORDER BY m.nim
20          ");
21
22          $search = '%' . $keyword . '%';
23
24          $stmt->execute([
25              'nim' => $search,
26              'nama' => $search
27          ]);
28
29          return $stmt->fetchAll();
30      }
31
32      $stmt = $this->pdo->query("
33          SELECT
34              m.*,
35              p.nama AS prodi_nama
36          FROM mahasiswa m
37          JOIN prodi p ON m.prodi_id = p.id
38          ORDER BY m.nim
39      ");
40
41      return $stmt->fetchAll();

```

```

app > repositories > MahasiswaRepository.php > ...
5 class MahasiswaRepository extends BaseModel
44 public function find(int $id): ?array
45 {
46     $stmt = $this->pdo->prepare("
47         SELECT *
48         FROM mahasiswa
49         WHERE id = :id
50     ");
51
52     $stmt->execute([
53         'id' => $id
54     ]);
55
56     $data = $stmt->fetch();
57
58     return $data ?: null;
59 }
60
61 1 reference | 0 overrides
62 public function create(array $data): int
63 {
64     $stmt = $this->pdo->prepare("
65         INSERT INTO mahasiswa
66         (nim, nama, email, prodi_id, angkatan)
67         VALUES
68         (:nim, :nama, :email, :prodi_id, :angkatan)
69     ");
70
71     $stmt->execute([
72         'nim' => $data['nim'],
73         'nama' => $data['nama'],
74         'email' => $data['email'],
75         'prodi_id' => $data['prodi_id'],
76         'angkatan' => $data['angkatan']
77     ]);

```

```

app > repositories > MahasiswaRepository.php > ...
5 class MahasiswaRepository extends BaseModel
80
81 1 reference | 0 overrides
82 public function update(int $id, array $data): bool
83 {
84     $stmt = $this->pdo->prepare("
85         UPDATE mahasiswa
86         SET
87             nim = :nim,
88             nama = :nama,
89             email = :email,
90             prodi_id = :prodi_id,
91             angkatan = :angkatan
92         WHERE id = :id
93     ");
94
95     return $stmt->execute([
96         'id' => $id,
97         'nim' => $data['nim'],
98         'nama' => $data['nama'],
99         'email' => $data['email'],
100         'prodi_id' => $data['prodi_id'],
101         'angkatan' => $data['angkatan']
102     ]);
103
104 1 reference | 0 overrides
105 public function delete(int $id): bool
106 {
107     $stmt = $this->pdo->prepare("
108         DELETE FROM mahasiswa
109         WHERE id = :id
110     ");
111
112     return $stmt->execute([
113         'id' => $id

```

10. Penggunaan Prepared Statement

Repository menggunakan prepared statement untuk menjalankan query yang membutuhkan parameter.

Contohnya pada proses pencarian:

```
$stmt = $this->pdo->prepare("
SELECT
    m.*,
    p.nama AS prodi_nama
FROM mahasiswa m
JOIN prodi p ON m.prodi_id = p.id
WHERE m.nim LIKE :nim
       OR m.nama LIKE :nama
ORDER BY m.nim
");
```

Kemudian parameter diberikan menggunakan:

```
$stmt->execute([
    'nim' => $search,
    'nama' => $search
]);
```

Prepared statement juga digunakan pada proses find(), create(), update(), dan delete().

Gambar 10. Prepared statement pada MahasiswaRepository

```

81 public function update(int $id, array $data): bool
82 {
83     $stmt = $this->pdo->prepare("
84         UPDATE mahasiswa
85         SET
86             nim = :nim,
87             nama = :nama,
88             email = :email,
89             prodi_id = :prodi_id,
90             angkatan = :angkatan
91         WHERE id = :id
92     ");
93
94     return $stmt->execute([
95         'id' => $id,
96         'nim' => $data['nim'],
97         'nama' => $data['nama'],
98         'email' => $data['email'],
99         'prodi_id' => $data['prodi_id'],
100         'angkatan' => $data['angkatan']
101     ]);
102 }
103

```

11. Controller Menampilkan Data Mahasiswa

Method `index()` pada `MahasiswaController` digunakan untuk menampilkan data mahasiswa.

```

public function index(): void
{
    $keyword = $_GET['search'] ?? "";

    $data = $this->repository->all($keyword);

    $this->view('mahasiswa/index', [
        'mahasiswa' => $data,
        'keyword' => $keyword
    ]);
}

```

Controller mengambil keyword pencarian, meminta data kepada Repository, kemudian mengirim data tersebut ke View.

Gambar 11. Method index pada MahasiswaController

```

13     public function index(): void
14     {
15         $keyword = $_GET['search'] ?? '';
16
17         $data = $this->repository->all($keyword);
18
19         $this->view('mahasiswa/index', [
20             'mahasiswa' => $data,
21             'keyword' => $keyword
22         ]);
23     }
24

```

12. Fitur Pencarian Mahasiswa

Pada halaman mahasiswa terdapat form pencarian berdasarkan NIM atau nama.

<form

action="/bkpm-webserver/acara10/public/mahasiswa"

method="GET">

Input pencarian menggunakan:

name="search"

Keyword tersebut kemudian diproses oleh Controller:

\$keyword = \$_GET['search'] ?? '';

Kemudian dikirimkan ke Repository:

\$data = \$this->repository->all(\$keyword);

Dengan demikian, pengguna dapat mencari data mahasiswa berdasarkan NIM atau nama.

Gambar 12. Fitur pencarian mahasiswa

Data Mahasiswa

+ Tambah Mahasiswa

Cari Reset

No	NIM	Nama	Email	Prodi	Angkatan	Aksi
1	2401002	Ani Wijaya	ani@email.com	Teknik Informatika	2024	Edit Hapus

13. Tampilan Data Mahasiswa

Halaman mahasiswa menampilkan data dari database dalam bentuk tabel.

Informasi yang ditampilkan meliputi:

- NIM
- Nama
- Email
- Prodi
- Angkatan
- Edit
- Hapus

Halaman juga menyediakan tombol **Tambah Mahasiswa** dan fitur pencarian.

Gambar 13. Tampilan data mahasiswa

No	NIM	Nama	Email	Prodi	Angkatan	Aksi	
1	2401002	Ani Wijaya	ani@email.com	Teknik Informatika	2024	Edit	Hapus
2	2401004	Sheryn Febrylia	e41250604@student.polije.ac.id	Teknik Informatika	2025	Edit	Hapus
3	2401007	Nila Agustin	2401007@student.polije.ac.id	Teknik Informatika	2026	Edit	Hapus
4	2401010	Dinda Rachmawati	2401010@student.polije.ac	Sistem Informasi	2025	Edit	Hapus
5	2401099	Mahasiswa API	2401099@student.polije.ac.id	Teknik Informatika	2026	Edit	Hapus

14. Pengujian Tambah Data Mahasiswa

Pengujian tambah data dilakukan dengan memilih tombol:

+ **Tambah Mahasiswa**

Form yang ditampilkan terdiri dari:

- NIM
- Nama
- Email
- Prodi ID
- Angkatan

Setelah seluruh data diisi, tombol **Simpan** digunakan untuk mengirim data menggunakan method POST.

Form menggunakan:

```
<form
    action="/bkpm-webserver/acara10/public/mahasiswa"
    method="POST">
```

Gambar 14. Form tambah data mahasiswa

Screenshot hasil: buka halaman tambah mahasiswa dan tampilkan form sebelum disimpan.

Setelah data berhasil disimpan, pengguna diarahkan kembali ke halaman data mahasiswa.

Gambar 15. Hasil penambahan data mahasiswa

Data Mahasiswa

+ Tambah Mahasiswa

Cari berdasarkan NIM atau nama...

Cari

Reset

No	NIM	Nama	Email	Prodi	Angkatan	Aksi
1	2401002	Ani Wijaya	ani@email.com	Teknik Informatika	2024	Edit Hapus
2	2401004	Sheryn Febrylia	e41250604@student.polije.ac.id	Teknik Informatika	2025	Edit Hapus
3	2401007	Nila Agustin	2401007@student.polije.ac.id	Teknik Informatika	2026	Edit Hapus
4	2401010	Dinda Rachmawati	2401010@student.polije.ac	Sistem Informasi	2025	Edit Hapus
5	2401099	Mahasiswa API	2401099@student.polije.ac.id	Teknik Informatika	2026	Edit Hapus

NIM

214578

Nama

Diana

Email

214578@polije.ac.id

Program Studi

TI - Teknik Informatika

Angkatan

2021

Kembali

Simpan

Data Mahasiswa

+ Tambah Mahasiswa

Cari berdasarkan NIM atau nama...

Cari

Reset

No	NIM	Nama	Email	Prodi	Angkatan	Aksi
1	214578	Diana	214578@polije.ac.id	Teknik Informatika	2021	Edit Hapus
2	2401002	Ani Wijaya	ani@email.com	Teknik Informatika	2024	Edit Hapus
3	2401004	Sheryn Febrylia	e41250604@student.polije.ac.id	Teknik Informatika	2025	Edit Hapus
4	2401007	Nila Agustin	2401007@student.polije.ac.id	Teknik Informatika	2026	Edit Hapus
5	2401010	Dinda Rachmawati	2401010@student.polije.ac	Sistem Informasi	2025	Edit Hapus
6	2401099	Mahasiswa API	2401099@student.polije.ac.id	Teknik Informatika	2026	Edit Hapus

15. Pengujian Edit Data Mahasiswa

Pengujian edit dilakukan dengan memilih tombol **Edit** pada salah satu data mahasiswa.

URL edit menggunakan format:

/bkpm-webserver/acara10/public/mahasiswa/edit/id

Form edit akan menampilkan data mahasiswa yang sudah ada sehingga pengguna dapat mengubah data.

Gambar 16. Form edit data mahasiswa

Screenshot hasil: klik tombol **Edit**, kemudian screenshot form yang menampilkan data mahasiswa sebelum diubah.

Setelah salah satu data diubah, tombol **Update** digunakan untuk menyimpan perubahan.

Gambar 17. Hasil perubahan data mahasiswa

Edit Data Mahasiswa

NIM

214578

Nama

Diana

Email

214578@polije.ac.id

Program Studi

SI - Sistem Informasi

Angkatan

2024

Kembali

Update

Data Mahasiswa + Tambah Mahasiswa

Cari berdasarkan NIM atau nama... Cari Reset

No	NIM	Nama	Email	Prodi	Angkatan	Aksi
1	214578	Diana	214578@polije.ac.id	Sistem Informasi	2024	Edit Hapus
2	2401002	Ani Wijaya	ani@email.com	Teknik Informatika	2024	Edit Hapus
3	2401004	Sheryn Febrylia	e41250604@student.polije.ac.id	Teknik Informatika	2025	Edit Hapus
4	2401007	Nila Agustin	2401007@student.polije.ac.id	Teknik Informatika	2026	Edit Hapus
5	2401010	Dinda Rachmawati	2401010@student.polije.ac	Sistem Informasi	2025	Edit Hapus
6	2401099	Mahasiswa API	2401099@student.polije.ac.id	Teknik Informatika	2026	Edit Hapus

Pada tombol hapus terdapat konfirmasi:

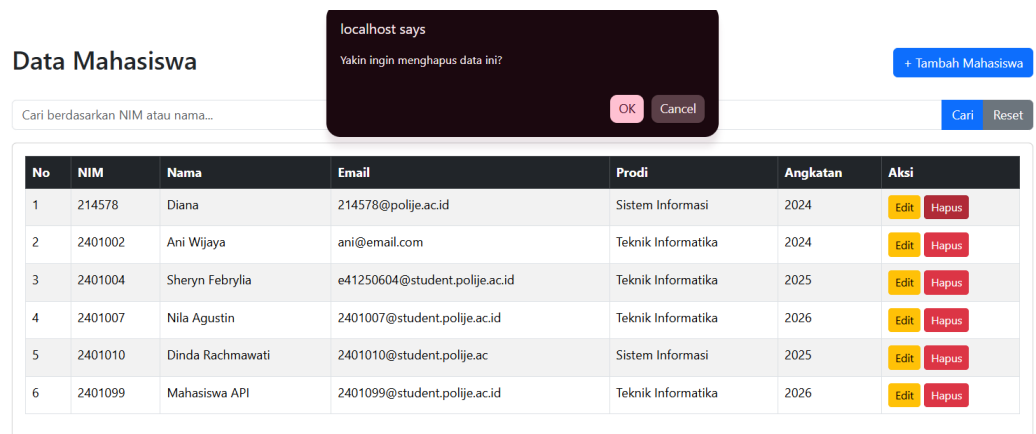
```
onsubmit="return confirm(
    'Yakin ingin menghapus data ini?'
);"
```

Konfirmasi tersebut digunakan untuk mencegah penghapusan data secara tidak sengaja.

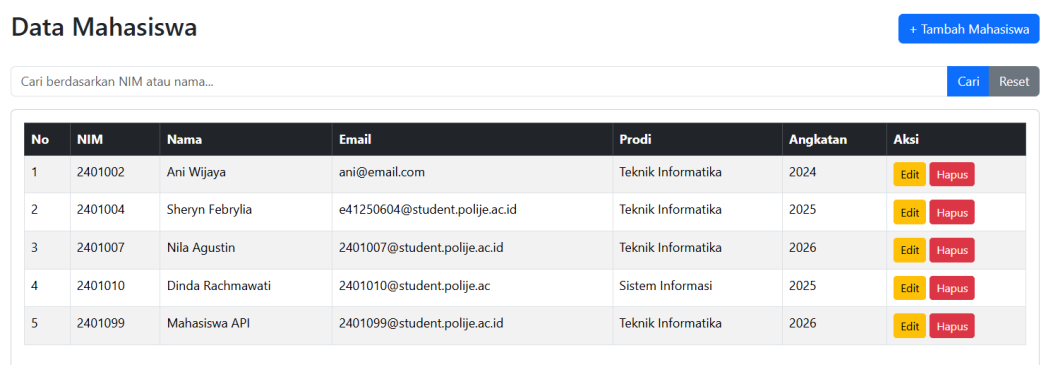
Setelah pengguna memilih OK, form dikirim menggunakan method POST ke URL:

/bkpm-webserver/acara10/public/mahasiswa/delete/id

Gambar 18. Konfirmasi penghapusan data mahasiswa



Gambar 19. Hasil penghapusan data mahasiswa



17. CRUD Data Program Studi

Selain mahasiswa, project Acara 10 juga memiliki pengelolaan data Program Studi.

ProdiController menggunakan ProdiRepository untuk melakukan proses:

all()

find()

create()

update()

delete()

Data Prodi terdiri dari:

- Kode Prodi
- Nama Prodi

Halaman Prodi juga memiliki tombol tambah, edit, dan hapus.

Gambar 20. Tampilan data Program Studi

Screenshot hasil: buka:

<http://localhost/bkpm-webserver/acara10/public/prodi>

Data Prodi

+ Tambah Prodi

No	Kode	Nama Prodi	Aksi
1	SI	Sistem Informasi	Edit Hapus
2	TI	Teknik Informatika	Edit Hapus
3	TK	Teknik Komputer	Edit Hapus

18. CRUD Data Mata Kuliah

Project juga menyediakan pengelolaan data mata kuliah.

MatakuliahController menggunakan:

```
private MatakuliahRepository $repository;
```

```
private ProdiRepository $prodiRepository;
```

Data mata kuliah terdiri dari:

- Kode Mata Kuliah
- Nama Mata Kuliah
- SKS
- Program Studi

Data program studi digunakan sebagai pilihan ketika menambahkan atau mengedit mata kuliah.

Gambar 21. Tampilan data mata kuliah

Screenshot hasil: buka:

<http://localhost/bkpm-webserver/acara10/public/matakuliah>

Data Mata Kuliah

+ Tambah Mata Kuliah

No	Kode	Nama Mata Kuliah	SKS	Prodi	Aksi	
1	SI105	Pengantar SI	2	Sistem Informasi	Edit	Hapus
2	TI101	Pemrograman Dasar	3	Teknik Informatika	Edit	Hapus
3	TI102	Basis Data	3	Teknik Informatika	Edit	Hapus

19. Inheritance pada Controller Lain

Selain MahasiswaController, class ProdiController dan MatakuliahController juga menggunakan Controller sebagai parent.

Strukturnya adalah:

BaseController



Controller



ProdiController



MatakuliahController

Dengan pola tersebut, Controller dapat menggunakan fungsi yang diwariskan dari BaseController melalui Controller.

Gambar 22. Inheritance pada Controller

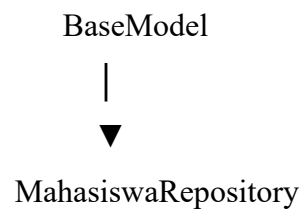
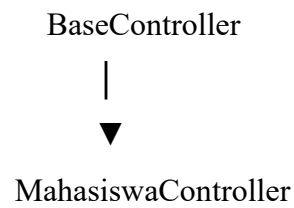
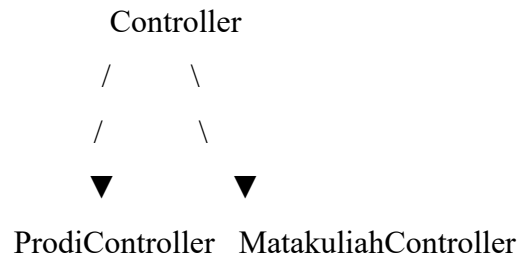
```
6      1 reference | 0 implementations
7      class ProdiController extends Controller
8      {
9          6 references
10         private ProdiRepository $repository;
```

20. Alur Inheritance pada Project

Penerapan inheritance pada project Acara 10 dapat digambarkan sebagai berikut:

BaseController





BaseController menyediakan fungsi umum seperti `view()` dan `redirect()`, sedangkan BaseModel menyediakan property PDO yang dapat digunakan oleh Repository.

Dengan demikian, class turunan dapat menggunakan fungsi dan property dari class induknya tanpa menulis ulang kode yang sama.

Pembahasan

Pada Acara 10 dilakukan penerapan konsep **inheritance** pada aplikasi MVC.

Inheritance memungkinkan sebuah class memperoleh property dan method dari class lain. Pada project ini, BaseController digunakan sebagai class dasar yang menyediakan method umum view() dan redirect().

Class Controller kemudian mewarisi BaseController. Selanjutnya beberapa Controller menggunakan Controller sebagai parent, sedangkan MahasiswaController menggunakan BaseController secara langsung.

Pada sisi pengelolaan database, dibuat BaseModel yang memiliki property \$pdo dengan visibility protected. MahasiswaRepository kemudian mewarisi BaseModel, sehingga Repository dapat menggunakan koneksi PDO yang diberikan melalui constructor.

Penerapan tersebut mengurangi duplikasi kode. Fungsi umum tidak perlu dibuat kembali pada setiap Controller atau Repository.

Selain konsep inheritance, project tetap menerapkan konsep sebelumnya seperti **Dependency Injection**, **Repository**, **CRUD**, dan **prepared statement**.

Dengan struktur tersebut, aplikasi menjadi lebih terorganisasi karena masing-masing class memiliki tanggung jawab yang jelas.

Kesimpulan

Berdasarkan praktikum Acara 10, dapat disimpulkan bahwa konsep **inheritance** dapat digunakan untuk membuat struktur aplikasi MVC yang lebih terorganisasi dan mengurangi pengulangan kode.

BaseController digunakan sebagai class dasar untuk menyediakan fungsi view() dan redirect(), sedangkan BaseModel digunakan sebagai class dasar yang menyimpan koneksi PDO.

Class turunan seperti MahasiswaController, ProdiController, MatakuliahController, dan MahasiswaRepository dapat menggunakan fungsi atau property yang diwariskan dari class induknya.

Selain penerapan inheritance, aplikasi juga berhasil menjalankan fungsi pengelolaan data seperti **menampilkan, mencari, menambah, mengedit, dan menghapus data mahasiswa**, serta menyediakan pengelolaan data **Prodi dan Mata Kuliah**.

Penerapan Base Class dan Inheritance membuat kode lebih mudah digunakan kembali, lebih terstruktur, dan lebih mudah dikembangkan untuk kebutuhan aplikasi berikutnya.